L B F A - C D

C

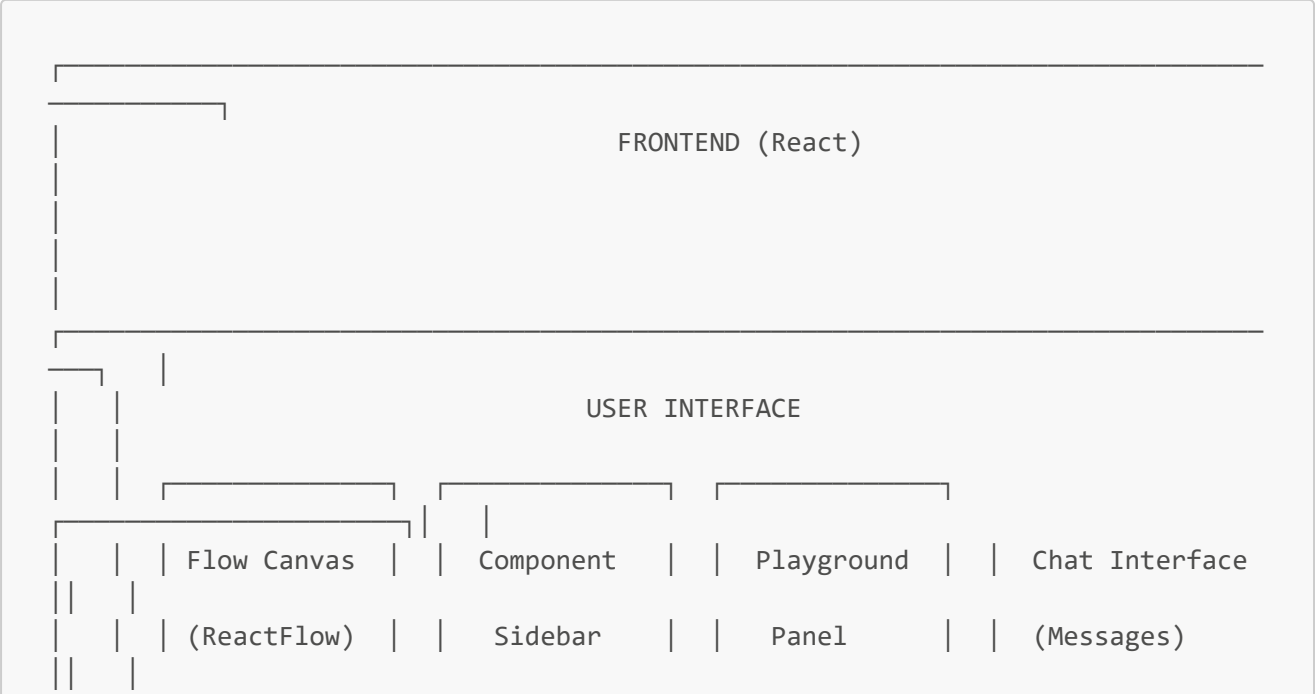
L B - AI

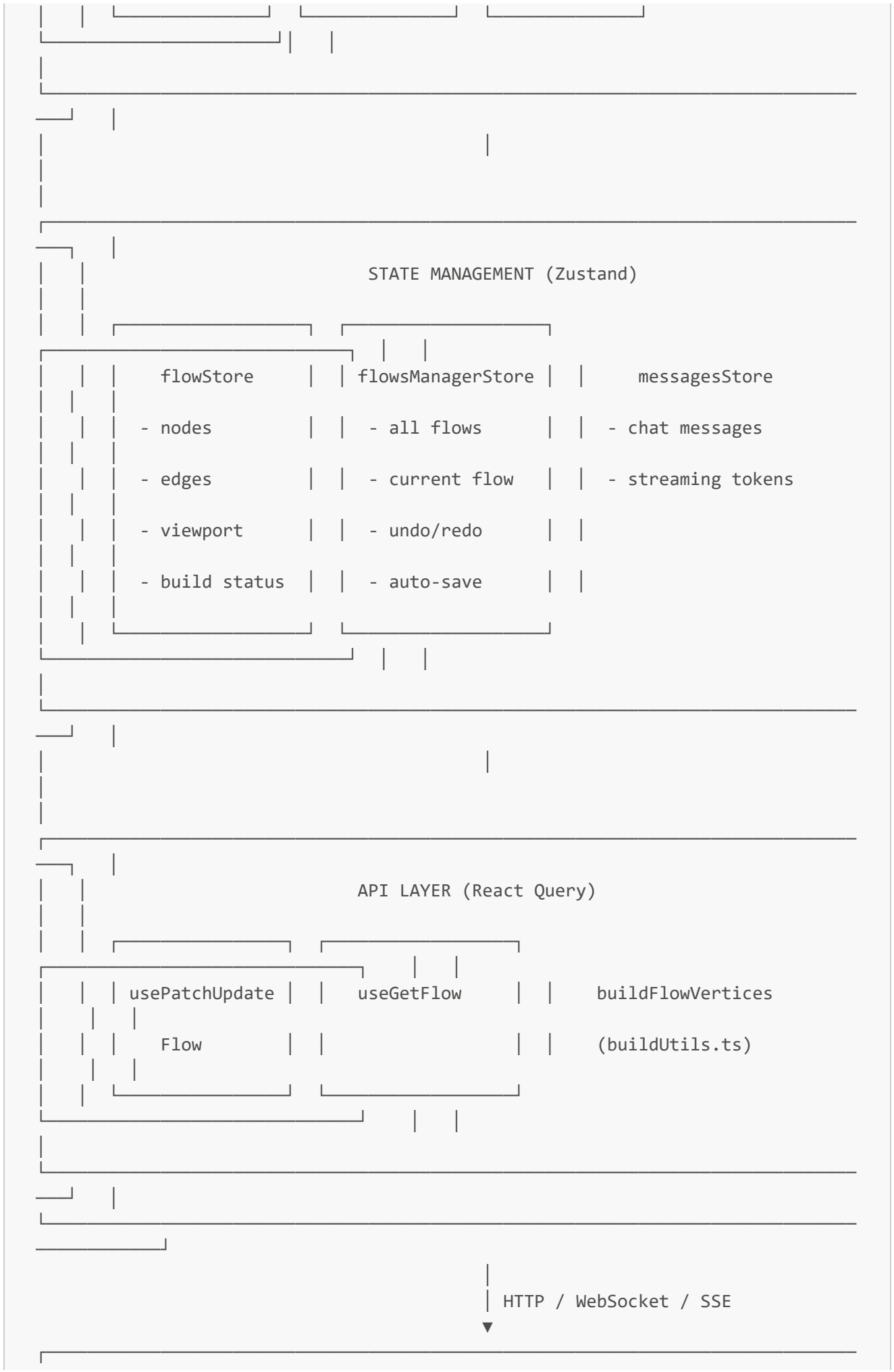
. :

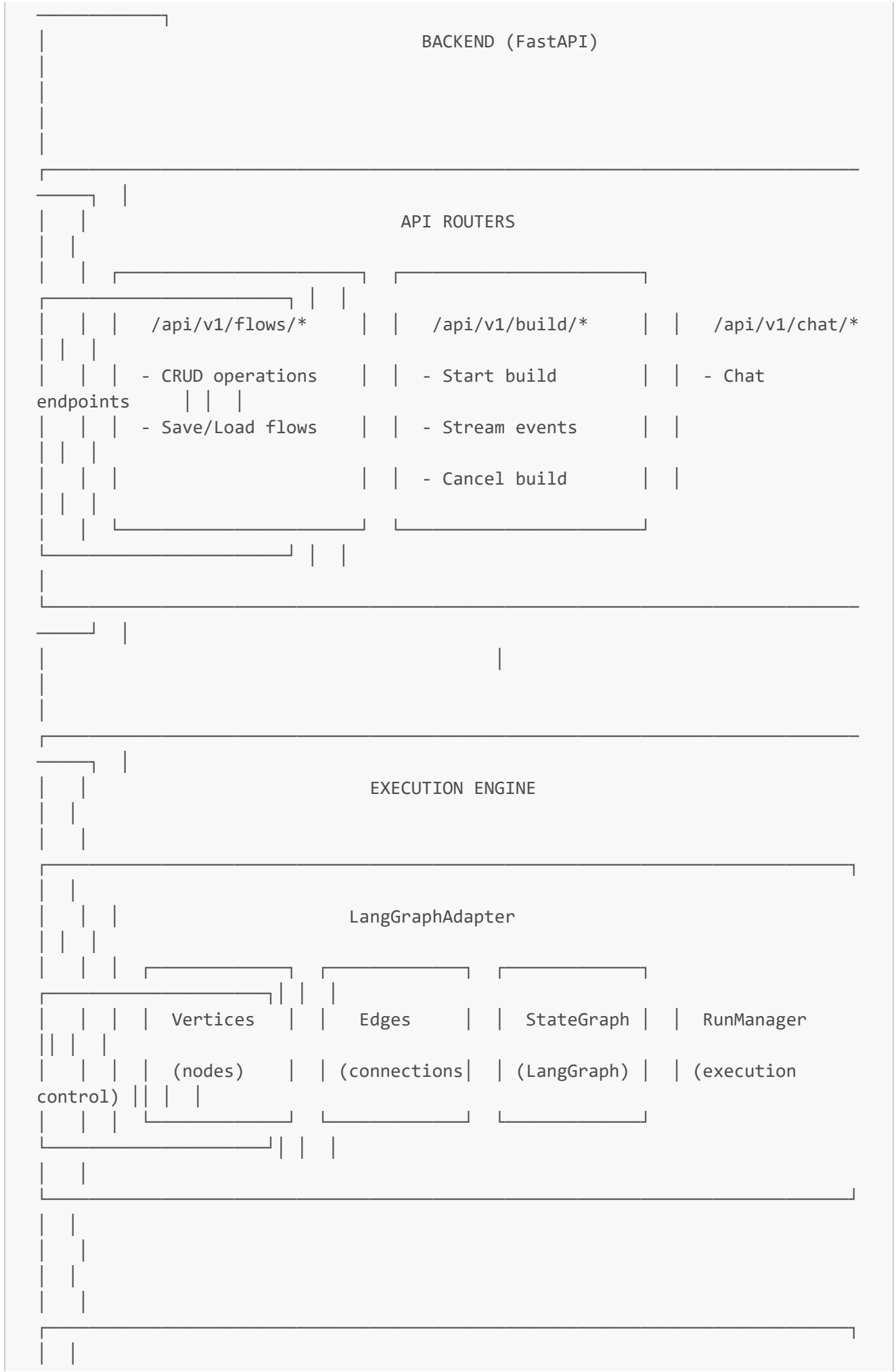
- **Frontend** : + + F
- **Backend** : F A I + L G
- **Database** : L / L
- **Execution Engine** : L G G

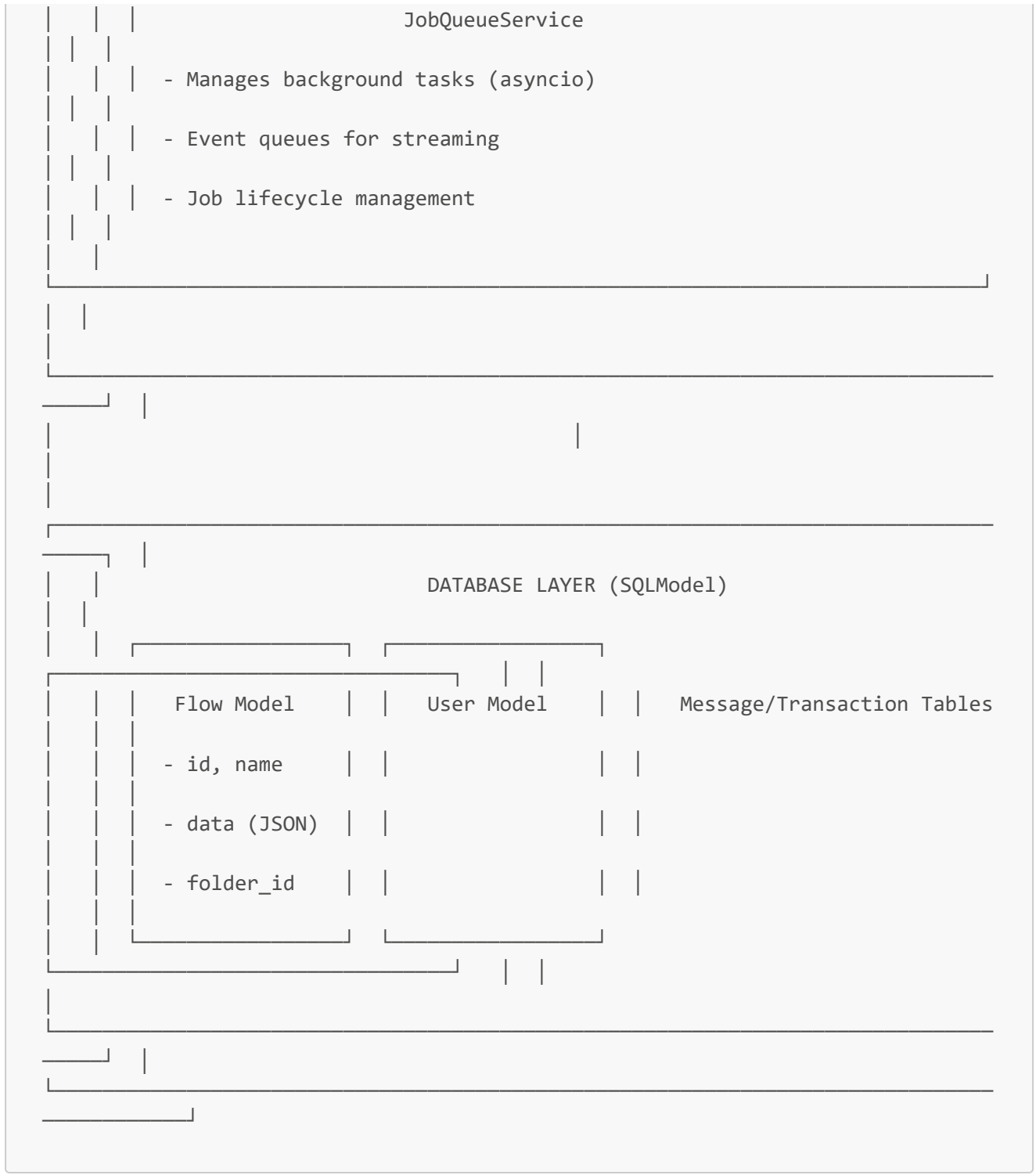
A D

H -L A

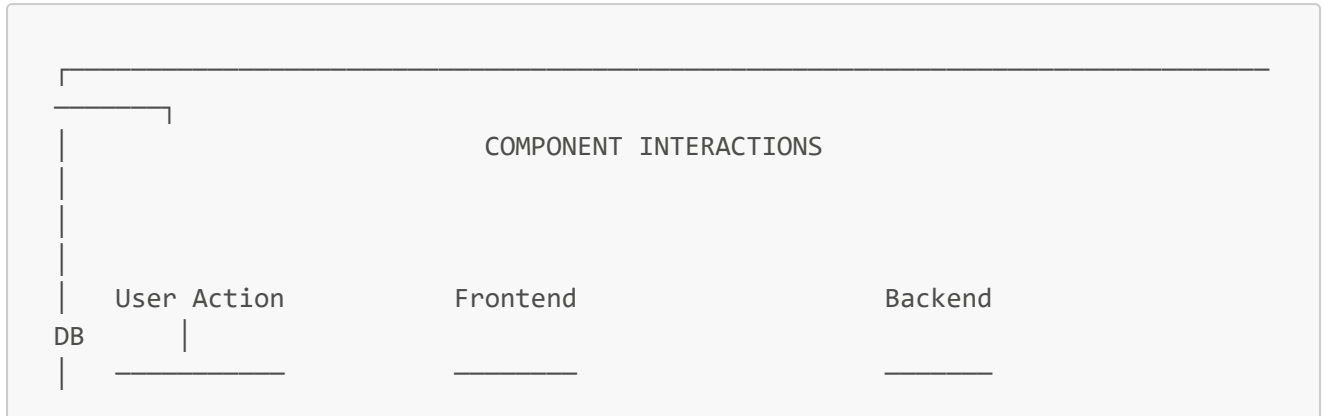


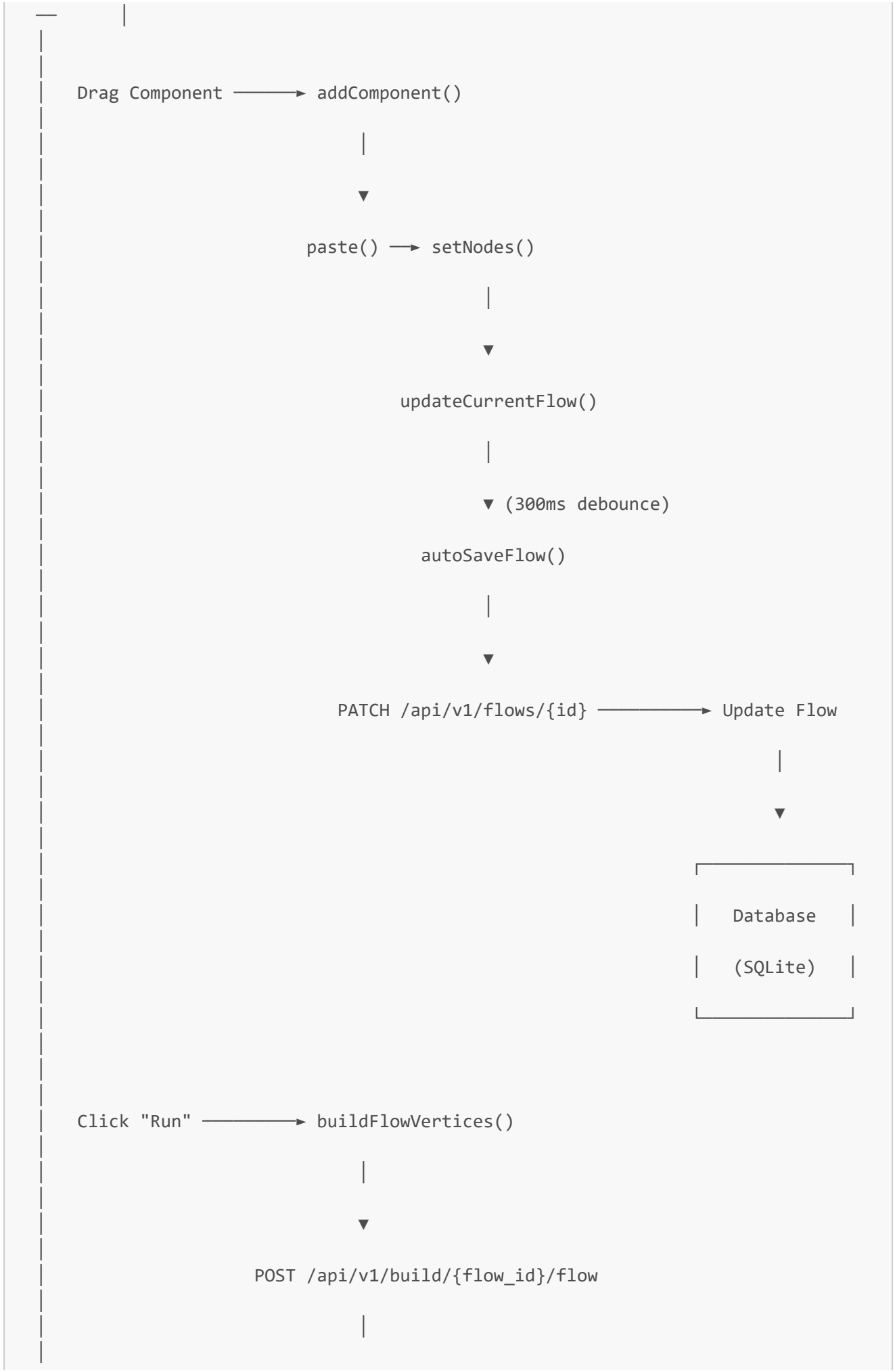

STATE MANAGEMENT (Zustand)
flowStore
- nodes
- edges
- viewport
- build status
flowsManagerStore
- all flows
- current flow
- undo/redo
- auto-save
messagesStore
- chat messages
- streaming tokens
API LAYER (React Query)
usePatchUpdate
Flow
useGetFlow
buildFlowVertices
(buildUtils.ts)
HTTP / WebSocket / SSE

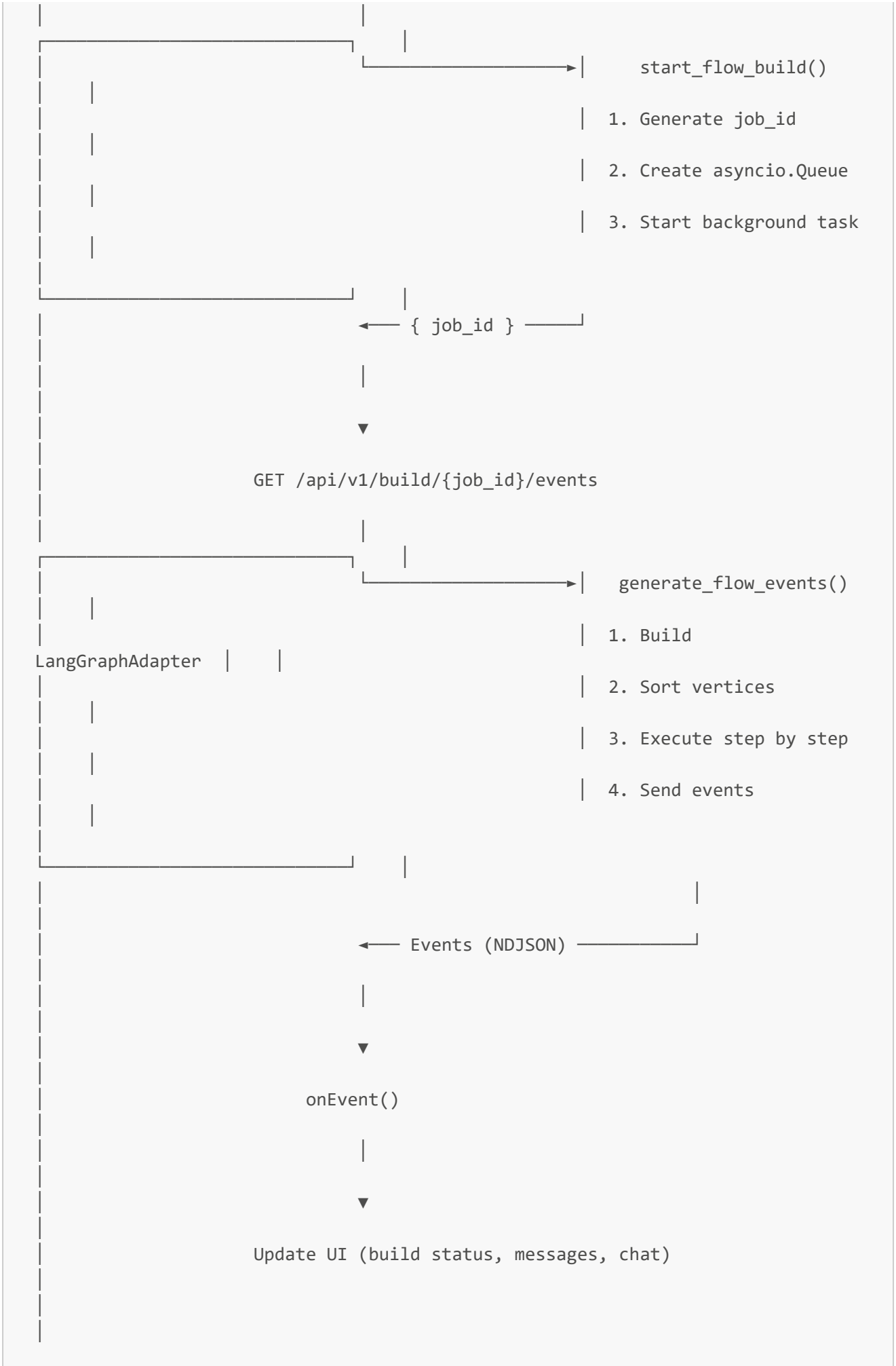

BACKEND (FastAPI)
API ROUTERS
/api/v1/flows/*
- CRUD operations
- Save/Load flows
/api/v1/build/*
- Start build
- Stream events
- Cancel build
/api/v1/chat/*
- Chat endpoints
EXECUTION ENGINE
LangGraphAdapter
Vertices
(nodes)
Edges
(connections)
StateGraph
(LangGraph)
RunManager
(execution control)


JobQueueService
- Manages background tasks (asyncio)
- Event queues for streaming
- Job lifecycle management
DATABASE LAYER (SQLModel)
Flow Model
- id, name
- data (JSON)
- folder_id
User Model
Message/Transaction Tables

C I D


COMPONENT INTERACTIONS
User Action
Frontend
Backend
DB

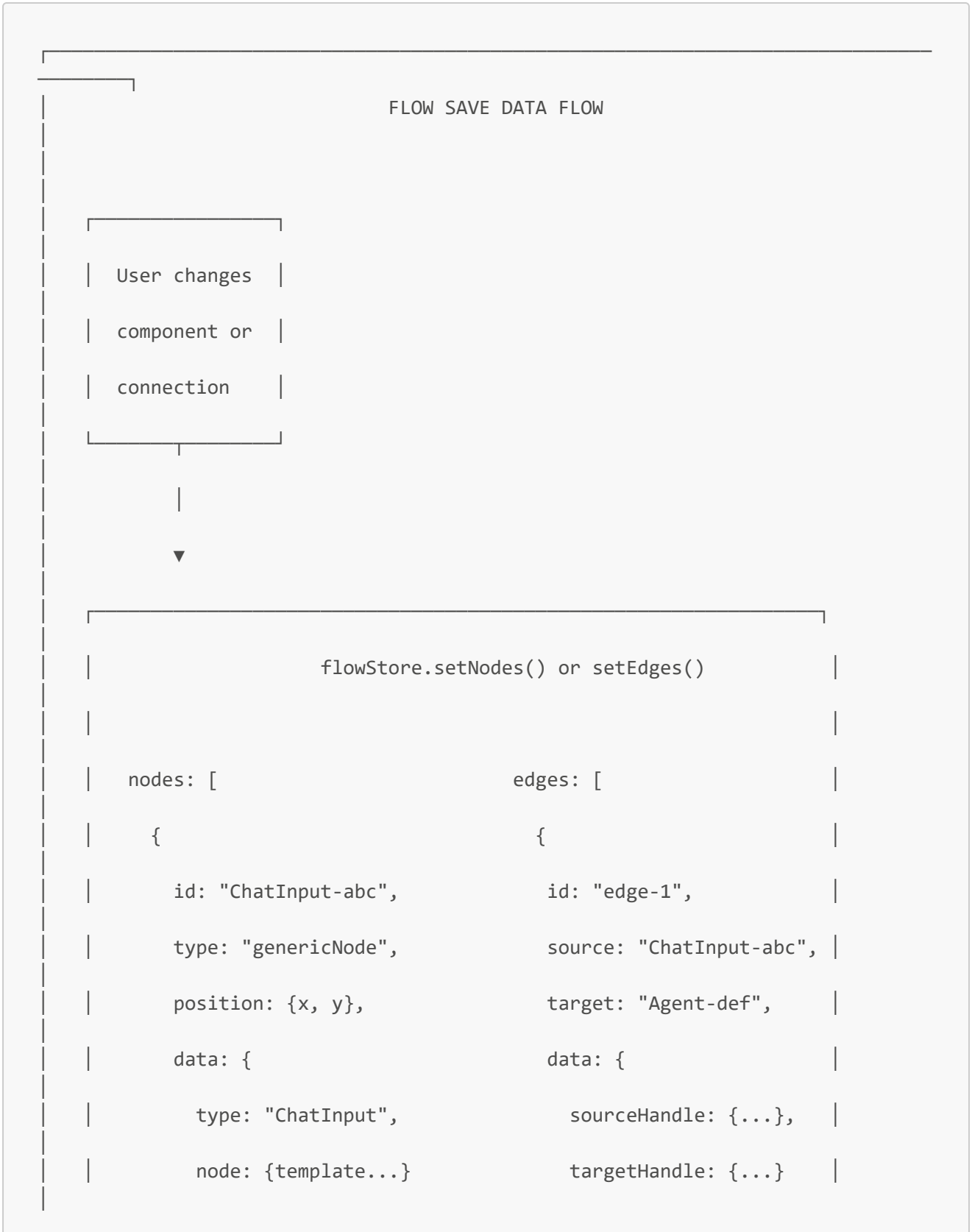

Drag Component
addComponent()
paste()
setNodes()
updateCurrentFlow()
(300ms debounce)
autoSaveFlow()
PATCH /api/v1/flows/{id}
Update Flow
Database
(SQLite)
Click "Run"
buildFlowVertices()
POST /api/v1/build/{flow_id}/flow


start_flow_build()
1. Generate job_id
2. Create asyncio.Queue
3. Start background task
{ job_id }
GET /api/v1/build/{job_id}/events
generate_flow_events()
1. Build
LangGraphAdapter
2. Sort vertices
3. Execute step by step
4. Send events
Events (NDJSON)
onEvent()
Update UI (build status, messages, chat)

```
└──────────────────────────────────────────────────────────────────────────────
──────┘
```

D F D

F D F


FLOW SAVE DATA FLOW
User changes component or connection
flowStore.setNodes() or setEdges()
nodes: [
{
id: "ChatInput-abc",
type: "genericNode",
position: {x, y},
data: {
type: "ChatInput",
node: {template...}
edges: [
{
id: "edge-1",
source: "ChatInput-abc",
target: "Agent-def",
data: {
sourceHandle: {...},
targetHandle: {...}

```
    }                                   }
  }                                   }
]                                   ]
```

```
        |
        ▼
```

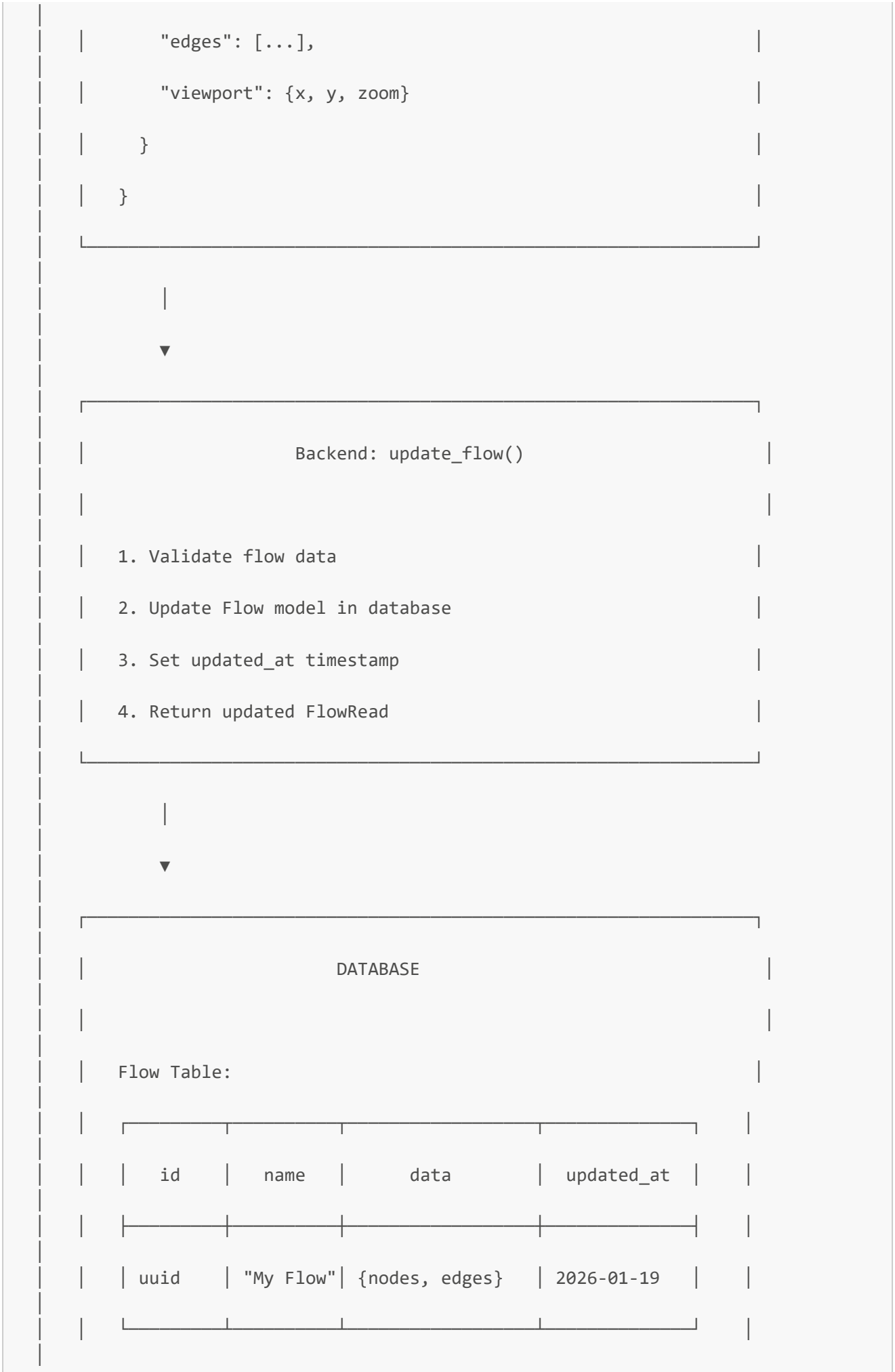
```
updateCurrentFlow({nodes, edges})

Updates flowsManagerStore.currentFlow.data
```

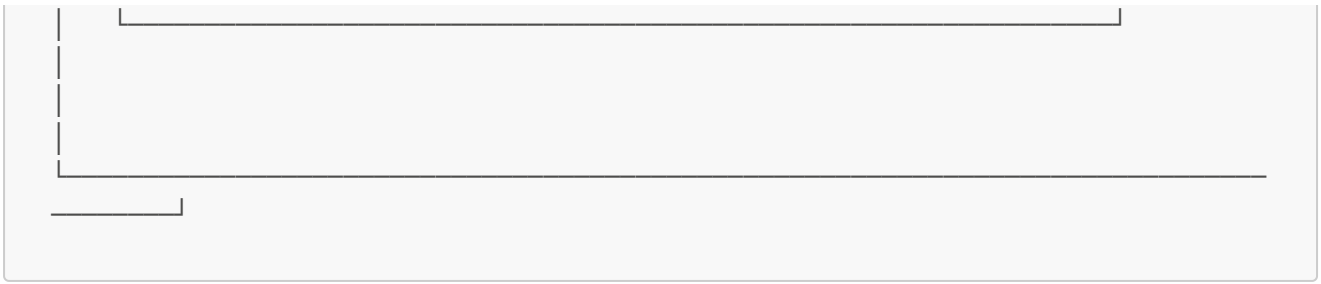
```
        |
        ▼
```

```
autoSaveFlow() [300ms debounce]

useDebounce(saveFlow, 300)
```

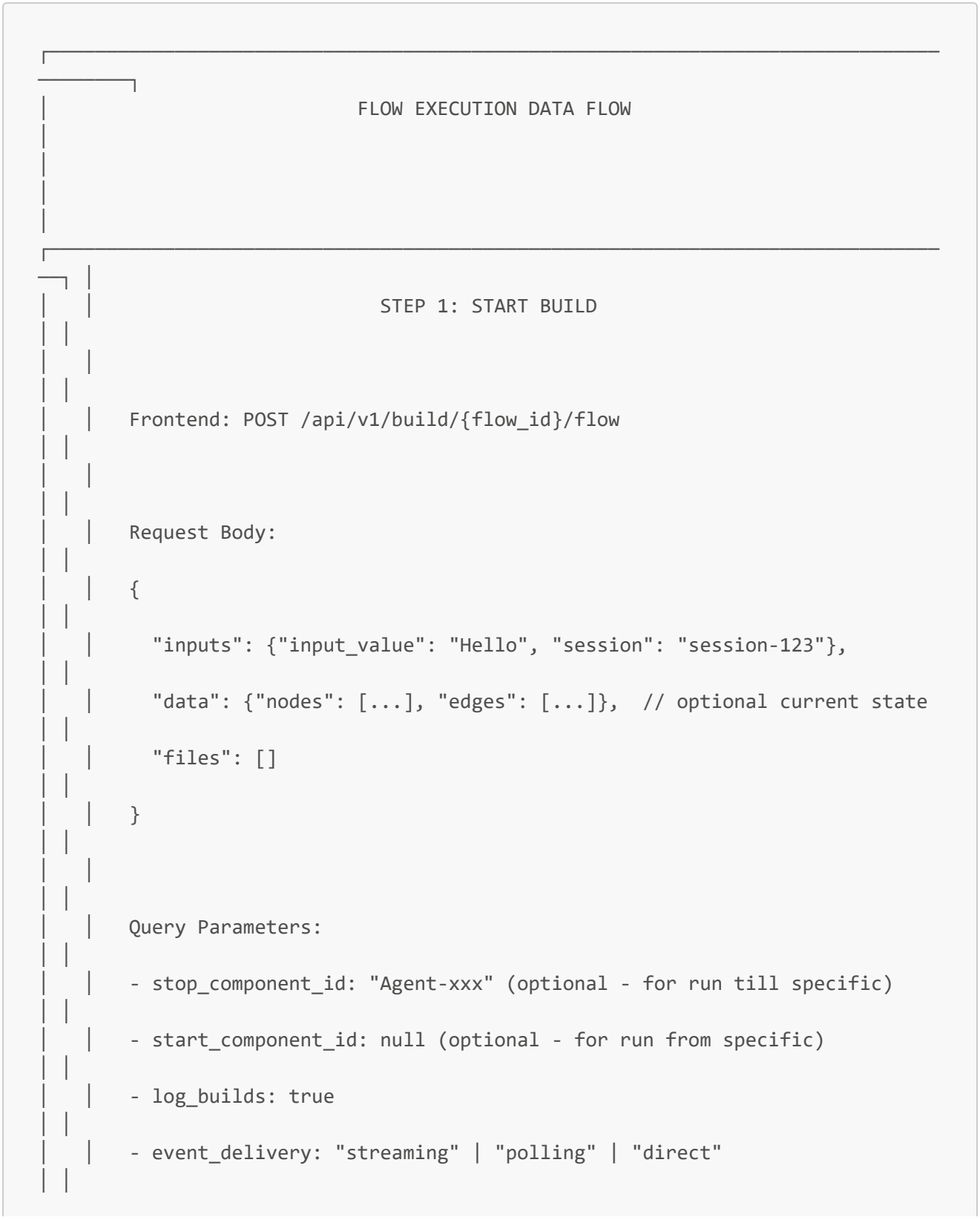
```
        |
        ▼
```

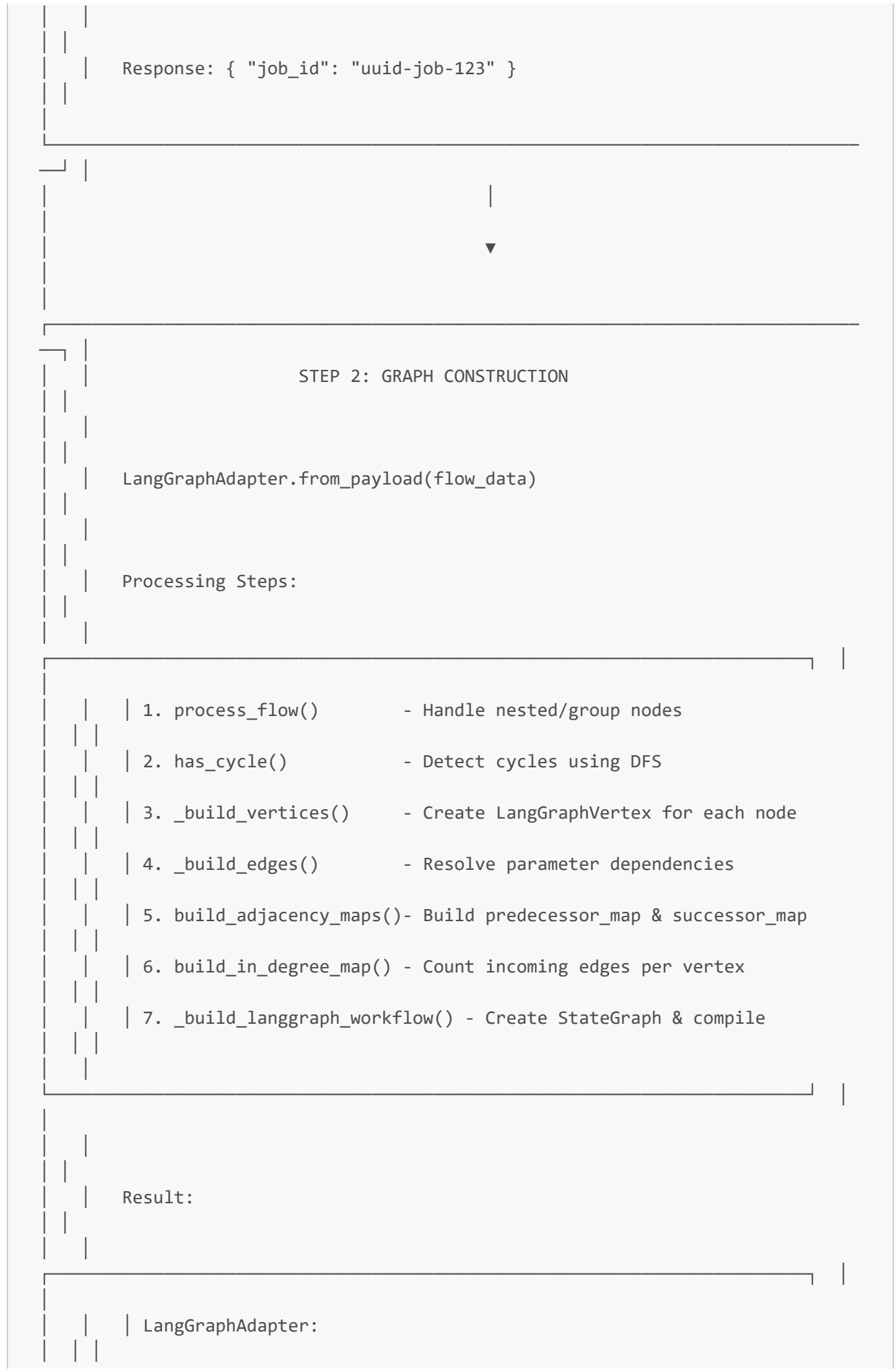
```
PATCH /api/v1/flows/{flow_id}

Request Body:
{
  "data": {
    "nodes": [...],
```


"edges": [...],
"viewport": {x, y, zoom}
}
}
Backend: update_flow()
1. Validate flow data
2. Update Flow model in database
3. Set updated_at timestamp
4. Return updated FlowRead
DATABASE
Flow Table:
id
name
data
updated_at
uuid
"My Flow"
{nodes, edges}
2026-01-19

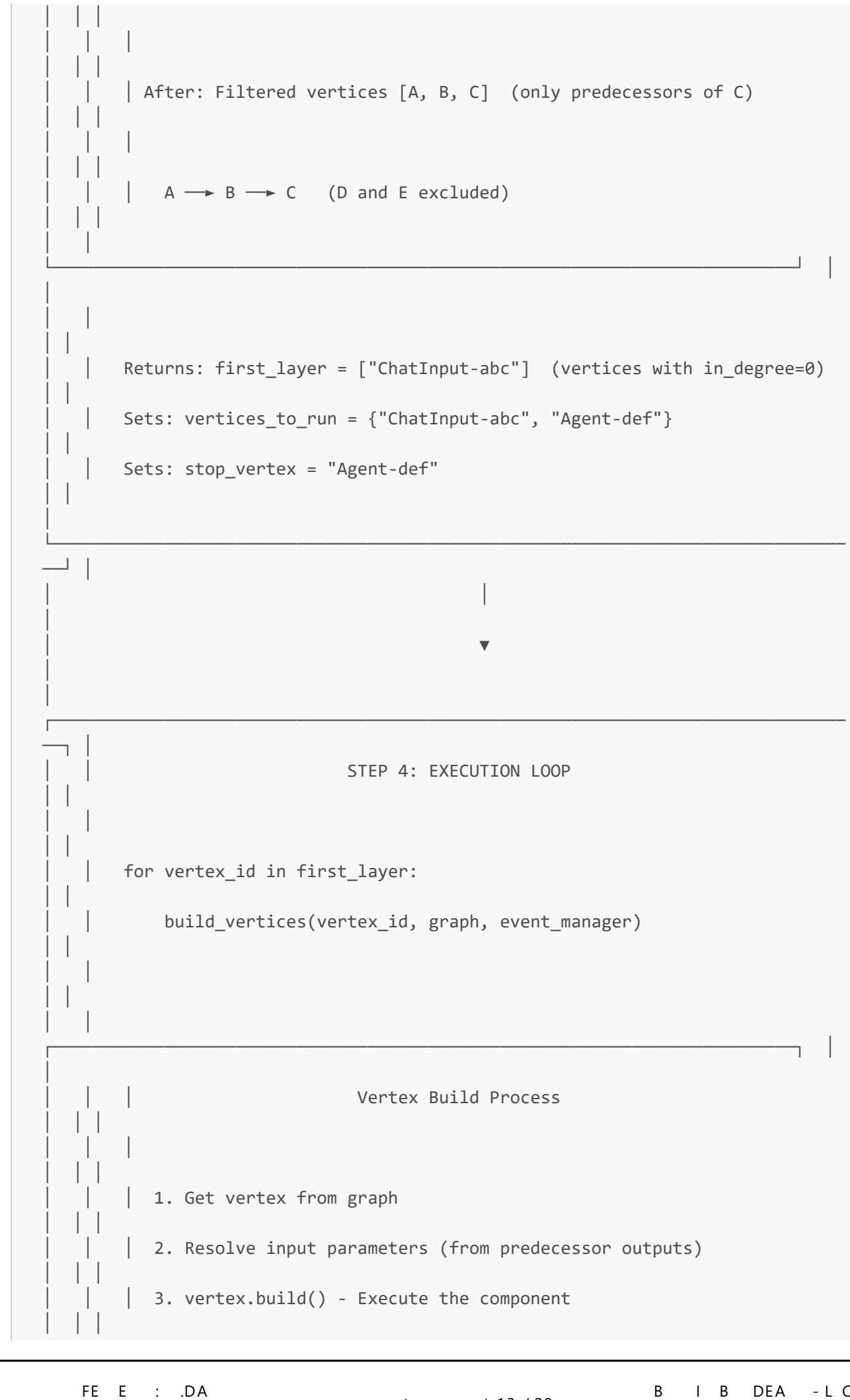


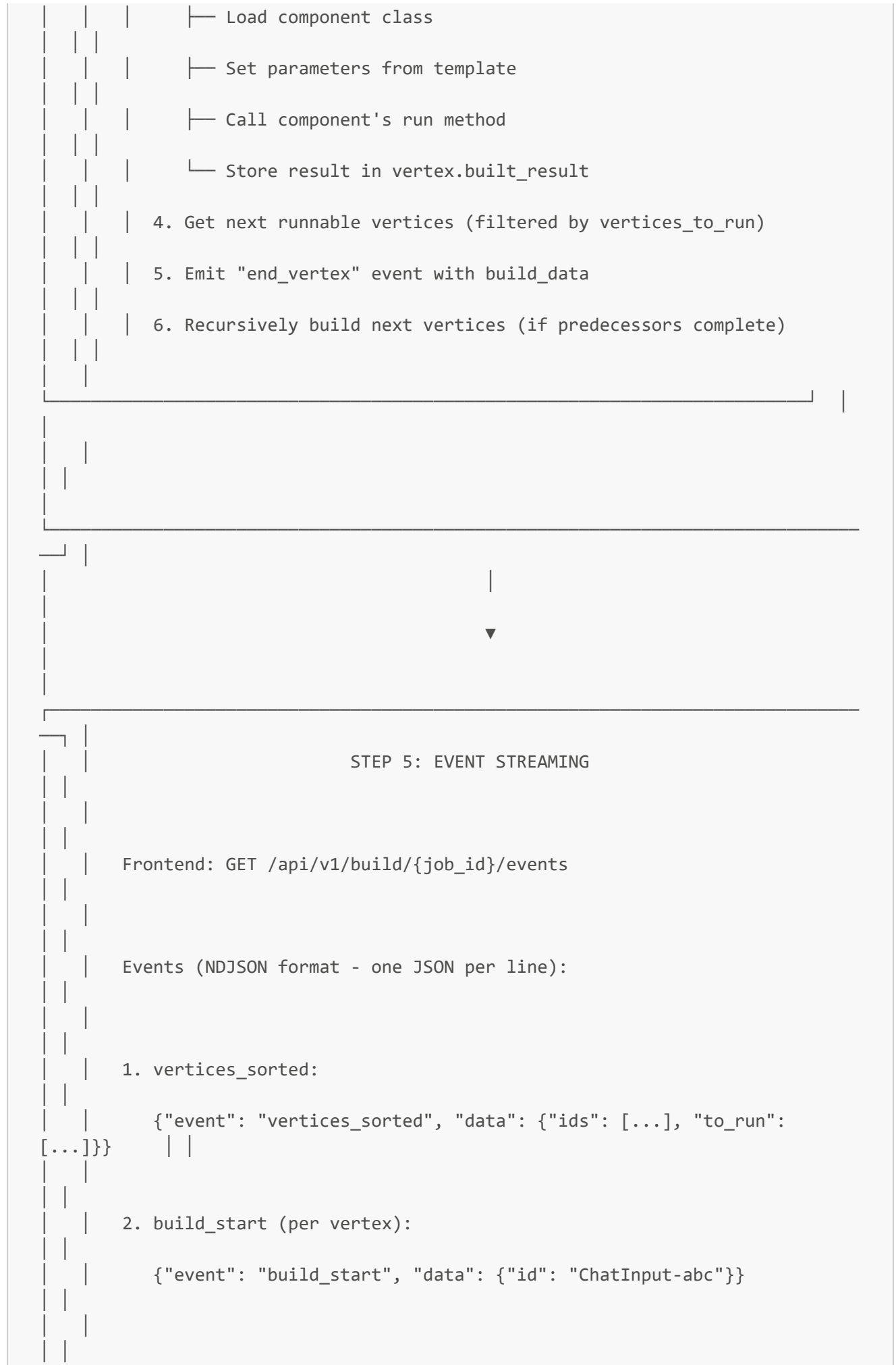
F E D F


FLOW EXECUTION DATA FLOW
STEP 1: START BUILD
Frontend: POST /api/v1/build/{flow_id}/flow
Request Body:
{
"inputs": {"input_value": "Hello", "session": "session-123"},
"data": {"nodes": [...], "edges": [...]}, // optional current state
"files": []
}
Query Parameters:
- stop_component_id: "Agent-xxx" (optional - for run till specific)
- start_component_id: null (optional - for run from specific)
- log_builds: true
- event_delivery: "streaming" | "polling" | "direct"


Response: { "job_id": "uuid-job-123" }
STEP 2: GRAPH CONSTRUCTION
LangGraphAdapter.from_payload(flow_data)
Processing Steps:
1. process_flow() - Handle nested/group nodes
2. has_cycle() - Detect cycles using DFS
3. _build_vertices() - Create LangGraphVertex for each node
4. _build_edges() - Resolve parameter dependencies
5. build_adjacency_maps()- Build predecessor_map & successor_map
6. build_in_degree_map() - Count incoming edges per vertex
7. _build_langgraph_workflow() - Create StateGraph & compile
Result:
LangGraphAdapter:


vertices: [LangGraphVertex, LangGraphVertex, ...]
vertex_map: {"ChatInput-abc": vertex, "Agent-def": vertex, ...}
edges: [{source, target, data}, ...]
predecessor_map: {"Agent-def": ["ChatInput-abc"], ...}
successor_map: {"ChatInput-abc": ["Agent-def"], ...}
in_degree_map: {"ChatInput-abc": 0, "Agent-def": 1, ...}
workflow: StateGraph (compiled)
STEP 3: VERTEX SORTING & FILTERING
sort_vertices(stop_component_id, start_component_id)
If stop_component_id provided:
filter_vertices_up_to_vertex()
Before: All vertices [A, B, C, D, E]


After: Filtered vertices [A, B, C] (only predecessors of C)
A ⟶ B ⟶ C (D and E excluded)
Returns: first_layer = ["ChatInput-abc"] (vertices with in_degree=0)
Sets: vertices_to_run = {"ChatInput-abc", "Agent-def"}
Sets: stop_vertex = "Agent-def"
STEP 4: EXECUTION LOOP
for vertex_id in first_layer:
build_vertices(vertex_id, graph, event_manager)
Vertex Build Process
1. Get vertex from graph
2. Resolve input parameters (from predecessor outputs)
3. vertex.build() - Execute the component


├── Load component class
├── Set parameters from template
├── Call component's run method
└── Store result in vertex.built_result
4. Get next runnable vertices (filtered by vertices_to_run)
5. Emit "end_vertex" event with build_data
6. Recursively build next vertices (if predecessors complete)
STEP 5: EVENT STREAMING
Frontend: GET /api/v1/build/{job_id}/events
Events (NDJSON format - one JSON per line):
1. vertices_sorted:
{"event": "vertices_sorted", "data": {"ids": [...], "to_run": [...]}}
2. build_start (per vertex):
{"event": "build_start", "data": {"id": "ChatInput-abc"}}

```
   3. end_vertex (per vertex):

      {"event": "end_vertex", "data": {"build_data": {

        "id": "ChatInput-abc",

        "valid": true,

        "data": {"results": {...}, "outputs": {...}},

        "next_vertices_ids": ["Agent-def"]

      }}}

   4. token (for streaming LLM output):

      {"event": "token", "data": {"id": "msg-123", "chunk": "Hello"}}

   5. add_message:

      {"event": "add_message", "data": {message object}}

   6. end:

      {"event": "end", "data": {}}
```

E F

E : C A F

B , ' :

- **Chat Input Agent** (G)
- **Text Input Chat Output**

Flow JSON Structure

```
{
  "name": "Chat with Agent",
  "id": "flow-uuid-123",
  "data": {
    "nodes": [
      {
        "id": "ChatInput-abc123",
        "type": "genericNode",
        "position": {"x": 100, "y": 200},
        "data": {
          "type": "ChatInput",
          "id": "ChatInput-abc123",
          "showNode": true,
          "node": {
            "display_name": "Chat Input",
            "template": {
              "input_value": {"type": "str", "value": ""},
              "sender": {"type": "str", "value": "User"},
              "sender_name": {"type": "str", "value": "User"}
            },
            "outputs": [{"name": "message", "types": ["Message"]}]
          }
        }
      },
      {
        "id": "Agent-uWrxN",
        "type": "genericNode",
        "position": {"x": 400, "y": 150},
        "data": {
          "type": "Agent",
          "id": "Agent-uWrxN",
          "node": {
            "display_name": "Agent",
            "template": {
              "model_provider": {"type": "str", "value": "Groq"},
              "model": {"type": "str", "value": "openai/gpt-oss-120b"},
              "groq_api_key": {"type": "str", "value": "***", "password":
true},
              "agent_instructions": {"type": "str", "value": "You are a helpful
assistant..."},
              "input": {"type": "str", "value": "Receiving input"}
            },
            "outputs": [{"name": "Response", "types": ["Message"]}]
          }
        }
      },
      {
        "id": "TextInput-xyz789",
        "type": "genericNode",
```

```
      "position": {"x": 600, "y": 200},
      "data": {
        "type": "TextInput",
        "id": "TextInput-xyz789",
        "node": {
          "display_name": "Text Input",
          "template": {
            "text": {"type": "str", "value": "Receiving input"}
          },
          "outputs": [{"name": "Output Text", "types": ["str"]}]
        }
      }
    },
    {
      "id": "ChatOutput-def456",
      "type": "genericNode",
      "position": {"x": 850, "y": 200},
      "data": {
        "type": "ChatOutput",
        "id": "ChatOutput-def456",
        "node": {
          "display_name": "Chat Output",
          "template": {
            "input_value": {"type": "str", "value": ""}
          },
          "outputs": []
        }
      }
    }
  ],
  "edges": [
    {
      "id": "edge-1",
      "source": "ChatInput-abc123",
      "target": "Agent-uWrxN",
      "data": {
        "sourceHandle": {
          "id": "ChatInput-abc123",
          "name": "message",
          "output_types": ["Message"]
        },
        "targetHandle": {
          "id": "Agent-uWrxN",
          "fieldName": "input",
          "inputTypes": ["Message", "str"]
        }
      }
    },
    {
      "id": "edge-2",
      "source": "Agent-uWrxN",
      "target": "TextInput-xyz789",
      "data": {
```

```
          "sourceHandle": {
            "id": "Agent-uWrxN",
            "name": "Response",
            "output_types": ["Message"]
          },
          "targetHandle": {
            "id": "TextInput-xyz789",
            "fieldName": "text",
            "inputTypes": ["str", "Message"]
          }
        }
      },
      {
        "id": "edge-3",
        "source": "TextInput-xyz789",
        "target": "ChatOutput-def456",
        "data": {
          "sourceHandle": {
            "id": "TextInput-xyz789",
            "name": "Output Text",
            "output_types": ["str"]
          },
          "targetHandle": {
            "id": "ChatOutput-def456",
            "fieldName": "input_value",
            "inputTypes": ["Message", "str"]
          }
        }
      }
    ],
    "viewport": {"x": 0, "y": 0, "zoom": 1}
  }
}
```

Graph Construction Result

```
predecessor_map:
{
  "ChatInput-abc123": [],           // No predecessors (input node)
  "Agent-uWrxN": ["ChatInput-abc123"],
  "TextInput-xyz789": ["Agent-uWrxN"],
  "ChatOutput-def456": ["TextInput-xyz789"]
}

successor_map:
{
  "ChatInput-abc123": ["Agent-uWrxN"],
  "Agent-uWrxN": ["TextInput-xyz789"],
  "TextInput-xyz789": ["ChatOutput-def456"],
  "ChatOutput-def456": []           // No successors (output node)
```

```
}

in_degree_map:
{
  "ChatInput-abc123": 0,    // First layer
  "Agent-uWrxN": 1,
  "TextInput-xyz789": 1,
  "ChatOutput-def456": 1
}

first_layer: ["ChatInput-abc123"]
```

Execution Order

```
Step 1: ChatInput-abc123
  - in_degree: 0, runs first
  - Outputs: Message("Hello, how are you?")
  - next_runnable: ["Agent-uWrxN"]

Step 2: Agent-uWrxN
  - Receives: input = Message("Hello, how are you?")
  - Calls Groq API with gpt-oss-120b
  - Outputs: Message("I'm doing well, thank you!")
  - next_runnable: ["TextInput-xyz789"]

Step 3: TextInput-xyz789
  - Receives: text = Message("I'm doing well, thank you!")
  - Outputs: "I'm doing well, thank you!" (as string)
  - next_runnable: ["ChatOutput-def456"]

Step 4: ChatOutput-def456
  - Receives: input_value = "I'm doing well, thank you!"
  - Displays in chat
  - next_runnable: [] (end)
```

A I

F C D E

Method	Endpoint	Description
POST	/api/v1/flows/	C
GET	/api/v1/flows/	L
GET	/api/v1/flows/{flow_id}	G
PATCH	/api/v1/flows/{flow_id}	(-)

Method	Endpoint	Description
DELETE	/api/v1/flows/{flow_id}	D

B /E E

Method	Endpoint	Description
POST	/api/v1/build/{flow_id}/flow	
GET	/api/v1/build/{job_id}/events	
POST	/api/v1/build/{job_id}/cancel	C

B

Parameter	Type	Description
stop_component_id		( )
start_component_id		
log_builds		
event_delivery		streaming polling , direct,

J

F C (/ / 1/ /)

```
interface FlowCreate {
  name: string;
  data: {
    nodes: NodeType[];
    edges: EdgeType[];
    viewport?: {x: number, y: number, zoom: number};
  };
  description?: string;
  folder_id?: string;
  is_component?: boolean;
  icon?: string;
  tags?: string[];
}
```

F (A CH / / 1/ /)

```
interface FlowUpdate {
  name?: string;
  data?: {
```

```
    nodes: NodeType[];
    edges: EdgeType[];
    viewport?: {x: number, y: number, zoom: number};
  };
  description?: string;
  folder_id?: string;
  locked?: boolean;
  access_type?: "PUBLIC" | "PRIVATE" | "PROTECTED";
}
```

```
interface NodeType {
  id: string;                // Unique ID: "{ComponentType}-{uuid}"
  type: "genericNode";       // Node renderer type
  position: {x: number, y: number};
  data: {
    type: string;            // Component type: "ChatInput", "Agent", etc.
    id: string;              // Same as node id
    showNode?: boolean;
    node: {
      display_name: string;
      template: Record<string, TemplateField>;
      outputs: OutputDefinition[];
      icon?: string;
    };
  };
}
```

E

```
interface EdgeType {
  id: string;
  source: string;          // Source node ID
  target: string;          // Target node ID
  sourceHandle?: string;
  targetHandle?: string;
  data: {
    sourceHandle: {
      id: string;
      name: string;
      output_types: string[];
      dataType: string;
    };
    targetHandle: {
      id: string;
      fieldName: string;
      inputTypes: string[];
```

```
      type: string;
    };
  };
}
```

B E

```
// vertices_sorted event
interface VerticesSortedEvent {
  event: "vertices_sorted";
  data: {
    ids: string[];        // All vertex IDs to execute
    to_run: string[];     // Vertices that will actually run
  };
}

// end_vertex event
interface EndVertexEvent {
  event: "end_vertex";
  data: {
    build_data: {
      id: string;
      valid: boolean;
      params: any;
      data: {
        results: Record<string, any>;
        outputs: Record<string, OutputValue>;
        logs: Record<string, LogEntry[]>;
        duration: string;
        timedelta: number;
      };
      next_vertices_ids: string[];
      inactivated_vertices: string[];
      top_level_vertices: string[];
    };
  };
}

// token event (streaming LLM output)
interface TokenEvent {
  event: "token";
  data: {
    id: string;           // Message ID
    chunk: string;        // Token text
  };
}

// end event
interface EndEvent {
  event: "end";
```

```
  data: {};
}
```

L G I

H F C L G

```
# 1. Create StateGraph with custom state
workflow = StateGraph(LangBuilderState)

# 2. Add nodes (each vertex becomes a node)
for vertex in vertices:
    node_func = create_node_function(vertex)
    workflow.add_node(vertex.id, node_func)

# 3. Add edges (connections between nodes)
for edge in edges:
    workflow.add_edge(edge.source, edge.target)

# 4. Set entry point (first vertex with no predecessors)
workflow.set_entry_point(first_layer[0])

# 5. Compile to get executable app
compiled_app = workflow.compile()
```

L B

```
class LangBuilderState(TypedDict):
    # Execution results
    vertices_results: dict[str, Any]   # {vertex_id: built_result}
    artifacts: dict[str, Any]          # {vertex_id: artifacts}
    outputs_logs: dict[str, dict]      # {vertex_id: logs}

    # Current context
    current_vertex: str
    completed_vertices: Annotated[list[str], add]  # Accumulates
    events: Annotated[list[dict], add]             # Accumulates

    # Flow metadata
    flow_id: str
    flow_name: str | None
    session_id: str
    user_id: str | None

    # Execution context
    event_manager: Any
    input_data: dict[str, Any]
```

```
    files: list[str] | None

    # Graph topology
    vertex_objects: dict[str, Any]
    predecessor_map: dict[str, list[str]]
    successor_map: dict[str, list[str]]
    in_degree_map: dict[str, int]
```

F C

```
def create_node_function(vertex: LangGraphVertex):
    """Wrap vertex in a LangGraph-compatible node function."""

    async def node_function(state: LangBuilderState) -> LangBuilderState:
        # 1. Resolve dependencies from predecessor results
        resolved_params = _resolve_vertex_dependencies(vertex, state)

        # 2. Update vertex params with resolved values
        vertex.update_raw_params(resolved_params, overwrite=True)

        # 3. Execute the component
        await vertex.build(
            user_id=state.get("user_id"),
            inputs=state.get("input_data", {}),
            event_manager=state.get("event_manager"),
        )

        # 4. Store results for downstream vertices
        state["vertices_results"][vertex.id] = vertex.built_result
        state["completed_vertices"].append(vertex.id)

        return state

    return node_function
```

C F

H I

" C "

.

```
Full Flow:
  ChatInput → Agent → TextInput → ChatOutput

Run Till "Agent":
  ChatInput → Agent ✓  (TextInput and ChatOutput excluded)
```

I D

1. Frontend Request

```
// buildUtils.ts
const buildUrl = `/api/v1/build/${flowId}/flow?stop_component_id=Agent-
uWrxN&log_builds=true`;

const response = await fetch(buildUrl, {
  method: "POST",
  body: JSON.stringify({
    inputs: { input_value: "Hello" },
    data: { nodes, edges }
  })
});
```

2. Backend Vertex Filtering

```
# utils.py
def filter_vertices_up_to_vertex(
    vertices_ids: list[str],
    stop_vertex_id: str,
    predecessor_map: dict[str, list[str]],
) -> set[str]:
    """Filter vertices to only include predecessors of stop vertex."""

    # Start with stop vertex
    filtered = {stop_vertex_id}
    queue = deque([stop_vertex_id])

    # BFS backwards through predecessors
    while queue:
        current = queue.popleft()
        for predecessor in predecessor_map.get(current, []):
            if predecessor not in filtered:
                filtered.add(predecessor)
                queue.append(predecessor)

    return filtered
```

3. Graph Sort with Filtering

```
# adapter.py
def sort_vertices(self, stop_component_id=None, start_component_id=None):
```

```
    """Sort and filter vertices."""

    # Store stop vertex for later use
    self.stop_vertex = stop_component_id

    # Filter vertices up to stop component
    first_layer, vertices_to_run_list, filtered =
get_sorted_vertices_for_langgraph(
        vertices_ids=all_vertex_ids,
        predecessor_map=self.predecessor_map,
        stop_component_id=stop_component_id,
    )

    # Update execution set
    self.vertices_to_run = filtered

    # Update run manager
    self.run_manager.build_run_map(
        predecessor_map={k: v for k, v in self.predecessor_map.items() if k in
filtered},
        vertices_to_run=self.vertices_to_run,
    )

    return first_layer
```

4. Next Vertex Filtering

```
# adapter.py
async def get_next_runnable_vertices(self, lock, vertex, cache=False):
    """Get next vertices, respecting vertices_to_run filter."""

    successors = self.successor_map.get(vertex.id, [])

    # Filter to only include vertices in vertices_to_run
    if self.vertices_to_run:
        successors = [s for s in successors if s in self.vertices_to_run]

    # ... rest of logic
```

E

```
Original Flow:
┌──────────────┐    ┌──────────────┐    ┌──────────────┐    ┌──────────────┐
│  ChatInput   │──→ │    Agent     │──→ │  TextInput   │──→ │  ChatOutput  │
│   (abc123)   │    │   (uWrxN)    │    │   (xyz789)   │    │   (def456)   │
└──────────────┘    └──────────────┘    └──────────────┘    └──────────────┘

After filter_vertices_up_to_vertex(stop="Agent-uWrxN"):
```

```
vertices_to_run = {"ChatInput-abc123", "Agent-uWrxN"}

┌──────────────┐     ┌──────────────┐
│  ChatInput   │────▶│    Agent     │  ✓ Executed
│   (abc123)   │     │   (uWrxN)    │
└──────────────┘     └──────────────┘

                                        ┌──────────────┐     ┌──────────────┐
                                        │  TextInput   │     │  ChatOutput  │  ✗
Excluded
                                        │   (xyz789)   │     │   (def456)   │
                                        └──────────────┘     └──────────────┘
```

K F

F

File	Purpose
src/frontend/src/stores/flowStore.ts	, ,
src/frontend/src/stores/flowsManagerStore.ts	, /
src/frontend/src/utils/buildUtils.ts	B / ,
src/frontend/src/controllers/API/queries/flows/*.ts	A I C D

B

File	Purpose
src/backend/base/langbuilder/api/v1/flows.py	F C [
src/backend/base/langbuilder/api/build.py	B /
src/backend/base/langbuilder/graph_langgraph/adapter.py	L G
src/backend/base/langbuilder/graph_langgraph/utils.py	G (,)
src/backend/base/langbuilder/services/database/models/flow/model.py	F
src/backend/base/langbuilder/services/job_queue/service.py	J

C I

Flow not saving : C -
Components not executing :
Run Till not working : E
Events not streaming : C C

D L

L :

- `SORT_VERTICES:` -
- `GET_NEXT_RUNNABLE:` -
- `BUILD_VERTICES:` -
- `GRAPH_STATE:` - G

Document Version: 1.0
Last Updated: January 19, 2026